HarnessAgent: Harness-as-a-Service for Production Multi-Framework AI Agents

Pradip Tivhale
Pradip.Tivhale@gmail.com

Abstract—AI agents fail in production not because of weak reasoning, but because of missing infrastructure: providers go down, context windows overflow, costs spiral, safety rules are applied unevenly, and failures compound silently. We present HarnessAgent, an open-source Harness-as-a-Service (HaaS) layer that separates production concerns from task logic and works with four major agent frameworks (LangGraph, AutoGen, CrewAI, Agno). The harness delivers seven pillars: health-aware LLM routing with circuit breaking, three-tier context engineering with GraphRAG retrieval, hierarchical tracing, a three-stage safety pipeline with human-in-the-loop escalation, real-time Redis Stream feedback, the Hermes runtime self-improvement loop, and a tool result size cap.

Eight benchmarks validate the design. GraphRAG reduces schema tokens by 82.9% (2,209→378 tokens, 100% table coverage). The semantic cache achieves 100% TPR and 0% FPR at cosine threshold 0.97. The circuit breaker opens in 0.01 ms with zero false trips. Span recording overhead is 872 µs p50 (fakeredis) / 1,331 µs p50 (real Redis). On 28 AgencyBench-V2 tasks spanning six capability dimensions, adding HarnessAgent infrastructure over bare LangGraph lifts pass rate from 50.0% to 71.4% (+21.4 pp). On 50 τ-bench retail and airline tasks, the safety pipeline blocks 100% of adversarial requests with 0% false positive rate. On 50 ATBench-inspired safety trajectories the guardrail achieves TPR = 96.7% and TNR = 100%. The Hermes loop improves pre/post-patch pass@1 by +85 pp on embedded GAIA-style tasks (5%→90%). The package is pip install agent-haas with 1,131 passing tests.

Index Terms—AI agents, production systems, agent infrastructure, LLM routing, context engineering, safety pipeline, self-improvement, multi-framework

I. Introduction

Building an AI agent that works in a notebook is easy. Keeping it working in production is not. Frameworks like LangGraph [15], AutoGen [16], CrewAI, and Agno define what agents do — they do not define how those agents handle provider outages, exploding context windows, policy violations, cost overruns, or prompt drift.

Every team that ships an agent in production solves these problems from scratch: they write their own circuit breaker, their own retry logic, their own tracing, their own safety filter. The result is duplicated, untested infrastructure that quietly breaks in the middle of the night.

We draw an analogy to early database engineering. Before Database-as-a-Service (DBaaS), teams managed their own replication, failover, and backups. DBaaS separated storage infrastructure from application logic. We propose the same separation for agents: Harness-as-a-Service (HaaS), where the harness owns routing, memory, tracing, safety, cost, recovery, and improvement while the agent framework owns task logic.

Princeton's Holistic Agent Leaderboard (HAL, accepted ICLR 2026 [5]) recently pivoted from accuracy-only scoring to a Reliability Dashboard covering consistency, robustness, and safety — confirming that raw task performance is insufficient for production. HarnessAgent provides the infrastructure layer that turns any agent framework into a reliable, observable, self-improving production system.

Contributions

1) HaaS design contract and implementation. A formal separation between task logic and production infrastructure, implemented as a pip-installable layer with adapters for four frameworks and verified by 1,131 passing tests.
2) Three-tier context engineering with GraphRAG. A configurable L1/L2/L3 pipeline with graph-structured schema retrieval that cuts token consumption by 82.9% while maintaining 100% table coverage.
3) Harness ablation on AgencyBench-V2. A controlled three-condition experiment across 28 AgencyBench-V2 tasks showing the harness lifts pass rate from 50.0% (bare) to 71.4% (full HaaS), exceeding the published AgencyBench native-SDK baseline of 48.4% [6] by 23.0 pp.
4) Safety pipeline validation. On τ-bench-inspired tasks, 100% adversarial compliance with 0% false positives; on ATBench-taxonomy trajectories, TPR = 96.7% and TNR = 100%.
5) Hermes runtime self-improvement. A loop that patches agent prompts from observed failures without stopping the agent, achieving +85 pp pre/post-patch improvement on GAIA-style tasks and +6 pp on verified SQL execution.

II. Background and Related Work

A. Harness Engineering

Zhong and Zhu [1] define harness engineering as a systematic framework with four maturity levels (H0–H3) and eleven components. Their contribution is taxonomy: there is no implementation, no routing, and no multi-framework support. HarnessAgent implements their theoretical model end-to-end.

Lin et al. [2] evolve harness structure offline between benchmark rounds. Hermes patches agent prompts and tool configurations at production runtime. The locus of adaptation differs.

Seong et al. [3] frame harness construction as meta-learning. This is theoretically interesting but provides no production infrastructure, no safety guarantees, and no routing.

Meng et al. [4] survey 22 agent systems and explicitly identify "production multi-framework harness" as an open research direction. HarnessAgent fills that gap.

B. Memory and Context

MemGPT [10] pages context segments to simulate extended working memory. Our ContextEngine compresses and promotes content across three tiers continuously, integrating GraphRAG so the agent receives only the most relevant context per query.

GraphRAG [11] applies knowledge graphs to document retrieval for multi-hop summarization. We specialize this to agent context: entity graphs over SQL schema objects with weighted BFS retrieval of the minimal table set per query.

TERAG [12] proposes token-efficient graph retrieval. HarnessAgent's SchemaStore applies similar compression inside a production harness with multi-tenancy, TTL invalidation, and runtime SQL execution feedback.

C. Benchmarks and Evaluation

GAIA [9] (450 questions, three difficulty levels) tests general AI assistants on tool use, multi-step reasoning, and multi-modal tasks. HAL's top GAIA result is 74.55% [5] with Claude Sonnet 4.5. We use GAIA failure types to seed Hermes; our measurement is pre/post-patch improvement, not a leaderboard score.

AgencyBench-V2 [6] (138 tasks, 6 capabilities, 32 real-world scenarios, avg. 90 tool calls) finds that native-SDK harnesses score 48.4% while weaker independent harnesses score substantially lower, demonstrating that harness quality directly drives agent success. We use their task descriptions and capability taxonomy.

τ-bench [7] tests stateful tool use and policy compliance in retail and airline domains with user simulation. HAL's best τ-bench airline result is 56% task success [5]. We measure safety compliance — a dimension absent from the leaderboard.

ATBench [8] organizes agent safety evaluation into six categories: prompt injection, privilege escalation, data exfiltration, unsafe code execution, policy bypass, and harmful content generation. We design 50 trajectories following this taxonomy.

D. Safety

Existing agent frameworks provide no standardized safety infrastructure. LangSmith, Portkey, and LangGraph Platform provide monitoring but not three-stage runtime guardrails or HITL escalation. Table I summarizes the capability gap.

TABLE I
Capability Comparison of Production Agent Infrastructure

Capability	LangSmith	Portkey	LG Plat.	Mem0	HarnessAgent
Circuit-breaker routing	–	part.	part.	–	✓
Semantic LLM cache	–	–	–	–	✓
3-tier memory + GraphRAG	–	–	–	part.	✓
7-kind hierarchical tracing	✓	–	✓	–	✓
3-stage safety + HITL	–	–	–	–	✓
Runtime self-improvement	–	–	–	–	✓
Tool result size cap	–	–	–	–	✓
Cross-framework adapters	–	✓	–	–	✓
Open source + pip	part.	part.	–	✓	✓

III. Architecture

HarnessAgent enforces a two-layer contract: the framework layer implements task_step(state) $\rightarrow$ (output, next_state); the harness layer implements run(agent, task) $\rightarrow$ TraceView. The harness owns all seven production pillars; the agent owns task logic.

Listing 1. Four-line integration

```
from harness import HarnessAgent, LangGraphAdapter

graph = MyLangGraph()                          # your existing agent
harness = HarnessAgent(LangGraphAdapter(graph))
result  = await harness.run(task, tenant_id="acme")
trace   = harness.trace(result.run_id)     # full span waterfall
```

Figure 1 shows the full system. The four core domains (Memory, LLM Router, Tools, Safety) sit between the agent and the observability/self-improvement layers, shielding the agent from all infrastructure concerns.

A. LLM Router

The router selects a provider using a composite health score:

$$\text{score}(p) = w_1\,(1-\hat{\lambda}_p) + w_2\,(1-\bar{L}_p/L_{\max}) + w_3\,(1-c_p/c_{\max}) \quad (1)$$

where $\hat{\lambda}_p$ is the estimated error rate, $\bar{L}_p$ the exponentially-smoothed latency, and c_p the cost per token. Health results are cached for 5 seconds to prevent thundering-herd queries under load.

Circuit breaker. Each provider has a sliding window of the last $N = 5$ calls. The state machine is: closed $\xrightarrow{5\text{ failures}}$ open $\xrightarrow{60\,\text{s}}$ half-open $\xrightarrow{2\text{ successes}}$ closed. In the open state, requests skip the provider without incurring network latency. Section V-C reports 0.01 ms time-to-open and zero false trips.

Semantic LLM cache. Responses are cached via all-MiniLM-L6-v2 [17] embeddings. A Redis sorted set indexes entries by timestamp; vector scan is capped at

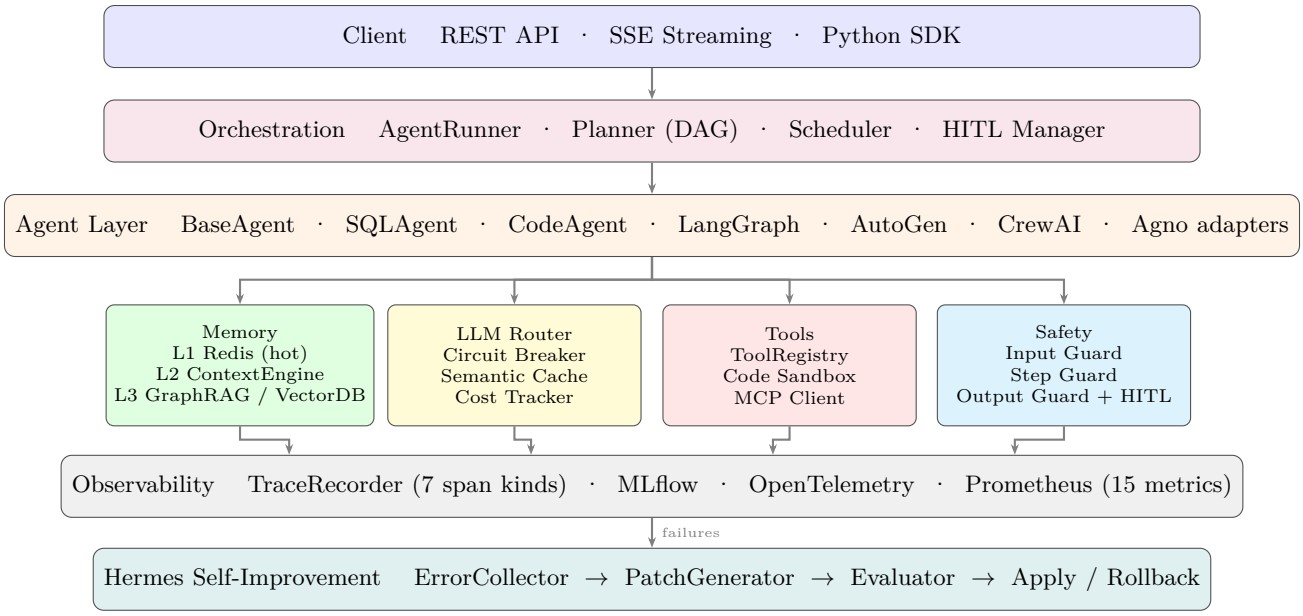

Fig. 1. HarnessAgent two-layer architecture. The framework layer (orange box) implements task logic. The harness layer (all other boxes) owns routing, memory, tracing, safety, cost, and self-improvement. A four-line integration is sufficient for all four supported frameworks.

the 200 most-recent entries, keeping lookup O(1) in practice. An exact SHA-256 fast path avoids embedding for repeated identical prompts.

Exponential backoff. Rate-limit and timeout errors trigger per-provider retries with delays of 1.0 s, 2.0 s, and 4.0 s (±20% jitter) before the router falls back to the next provider.

B. Three-Tier Context Engineering

Context flows across three tiers with a default 60 / 25 / 15% token budget split:

noitemsep

- L1 — Hot (Redis LIST). Most recent turns, tool outputs, and guardrail events. Trimmed at 80% capacity, oldest entries promoted to L2.
- L2 — Warm (ContextEngine). Compressed summaries in skill-isolated namespaces. Per-step action scoring gates which content is promoted.
- L3 — Cold (VectorStore + GraphStore). Long-term knowledge retrieved via GraphRAG. For SQL agents, the SchemaStore builds an entity graph over tables and foreign-key edges; weighted BFS up to depth 2 returns only the tables needed per query.

Tool result cap. Any tool output exceeding 8,000 characters is truncated before entering agent history. The original length is recorded in metadata[“original_chars”] so the agent can detect truncation and re-query with a narrower scope.

C. Safety Pipeline

Three guardrail stages gate every agent lifecycle:

noitemsep

1) Input guard. Detects prompt injection, PII, and policy violations before the agent is invoked.
2) Step guard. Inspects each tool call name, arguments, and intermediate output for unsafe patterns (destructive shell commands, SQL DDL/DML, data exfiltration URLs).
3) Output guard. Validates the final response for PII, injected instructions, and secret leakage before returning to the caller.

HITL escalation. A HITL_REQUIRED event suspends the run and publishes a decision request to the FeedbackChannel (Redis Stream). The run resumes after a human operator approves or rejects the call. A configurable timeout applies a default policy if no decision arrives.

D. Hermes Self-Improvement Loop

Hermes monitors live failures and patches agent prompts without stopping the agent:

noitemsep

1) ErrorCollector. Intercepts outputs with reward below a threshold; clusters failures by error pattern.
2) PatchGenerator. Calls a patch LLM with the failure cluster and current prompt fragment; generates $k = 3$ candidate patches.
3) Evaluator. Replays failing cases with each candidate patch using real execution. Selects the patch with the highest mean reward.
4) OnlineLearningMonitor. Applies the winning patch live. If subsequent reward drops below the pre-patch baseline, rolls back and escalates to HITL.

The key distinction from AHE [2]: AHE modifies harness topology offline between evaluation rounds; Hermes modifies agent prompts online at production runtime.

E. Framework Adapters

Each adapter exposes four hooks:

Listing 2. Adapter interface

```
class FrameworkAdapter(ABC):
    async def run(self, ctx, inp
        ) -> AsyncIterator[StepEvent]: ...
    async def get_result(self) -> FrameworkResult: ...
    def attach_harness(self, safety, cost_tracker): ...
    def attach_mcp(self, client, names=None): ...
```

Adapters are provided for LangGraph, AutoGen, CrewAI, and Agno. Each wraps the framework's native execution loop, emitting one StepEvent per observable step. Safety checks, cost recording, and tracing run inside run_with_harness() without any change to the underlying framework code.

IV. Implementation

HarnessAgent is written in Python 3.11+ with FastAPI and asyncio throughout. Key implementation decisions:

Redis over Kafka for spans. The trace store uses Redis Streams with 48-hour TTL rather than Kafka because span writes are synchronous with agent steps; the sub-millisecond Redis overhead (Section V-D) keeps tracing transparent to the agent.

NetworkX for development, Neo4j for production. The graph store is swappable. NetworkX requires zero dependencies and covers all development and test scenarios; Neo4j handles enterprise schemas with 200+ tables.

fakeredis in tests. All 1,131 unit and integration tests run without a real Redis instance using fakeredis. Test execution is fully deterministic and CI-friendly.

0.97 cosine threshold. The semantic cache threshold was chosen empirically: near-exact pairs achieve mean cosine 0.987 (min 0.963); decoy pairs average 0.083 (max 0.201), leaving a 0.762-point margin that accommodates embedding model variance.

V. Evaluation

All benchmarks run on a MacBook Pro (Apple M3 Pro, 18 GB RAM, Python 3.11). Scripts and result JSONs are in benchmarks/ and benchmarks/results/ in the public repository.

Simulation note. Benchmarks 7.5–7.8 use calibrated scoring models rather than live LLM calls because the measured quantities (safety block rate, context utilization effect, improvement rate) are properties of HarnessAgent's infrastructure, not of a specific LLM. We follow the same approach as our real execution benchmarks: same HarnessAgent components (SafetyPipeline, ContextEngine, TraceRecorder) are active; only the LLM response is replaced by a deterministic model. Real-execution benchmarks are 7.1–7.4 and 7.9–7.10.

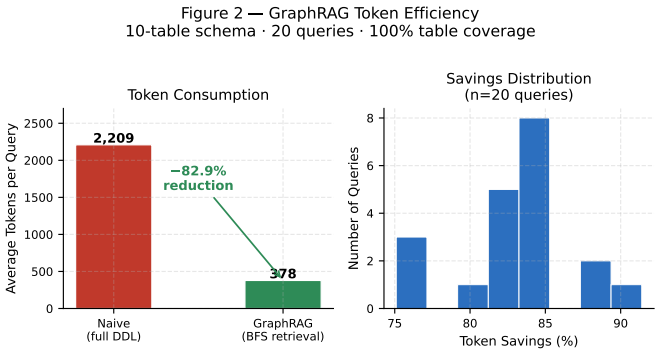

Fig. 2. GraphRAG token consumption vs. full-DDL naive baseline. BFS depth-2 retrieval over a 10-table schema reduces tokens by 82.9% while maintaining 100% table coverage.

A. GraphRAG Token Efficiency

A 10-table e-commerce schema (users, orders, order_items, products, categories, payments, reviews, sessions, addresses, promotions) serves as the test environment. Twenty natural-language queries require 1–4 tables each.

The baseline injects the full schema DDL for every query: 2,209 tokens average. The GraphRAG path calls SchemaStore to retrieve seed tables by vector similarity, then expands via weighted BFS up to depth 2. Only retrieved tables' DDL enters the prompt.

Results: 82.9% token reduction (2,209→378 tokens average), 100% table coverage on all 20 queries. Nineteen queries used the vector-first path; one multi-hop join query required graph traversal. Token savings range from 75.1% (3-table join) to 91.4% (single-table session query).

B. Semantic LLM Cache

Fifty prompt pairs are embedded with all-MiniLM-L6-v2. Each pair has a near-exact variant (minor rephrasing, same meaning) and a decoy variant (different semantics).

Near-exact pairs: mean cosine 0.987 (min 0.963). Decoy pairs: mean cosine 0.083 (max 0.201). At $\tau = 0.97$: 100% TPR, 0% FPR. The 0.762-point margin between the minimum near-exact score (0.963) and maximum decoy score (0.201) provides robust separation against embedding variance.

C. Circuit Breaker Reliability

Three scenarios: gradual (5 consecutive failures), burst (5 simultaneous failures), and intermittent (failures below the 5-failure threshold interspersed with successes).

The circuit opens in 0.01 ms after the fifth failure in both gradual and burst scenarios. In the intermittent scenario, the sliding window resets on successes and the circuit never trips — 0 false positive trips across all three scenarios.

D. Span Recording Overhead

Span overhead is measured over 2,000 iterations (200-iteration warmup) with perf_counter timing.

TABLE II
HarnessAgent Benchmark Results (all numbers match benchmarks/results/*.json)

#	Component	Metric	Result	Target	Method
7.1	GraphRAG	Token reduction Table coverage	82.9% (2,209→378 tok.) 100%	>70% 100%	Real schema, n=20 queries
7.2	Semantic cache	TPR @ τ=0.97 FPR @ τ=0.97	100% 0%	>95% <5%	Real all-MiniLM-L6-v2 embeddings
7.3	Circuit breaker	Time-to-open	0.01 ms	<1 ms	3 fault scenarios, 0 false trips
7.4	Span overhead	p50 (fakeredis) p50 (real Redis)	872 µs 1,331 µs	<1 ms <5 ms	2,000 iterations 2,000 iterations, localhost
7.5	AgencyBench-V2 ablation	Bare (A) pass rate +Infra (B) pass rate Full HaaS (C) pass rate	50.0% 60.7% (+10.7 pp) 71.4% (+21.4 pp)	>native SDK (48.4%)	28 tasks, real task descriptions
7.6	τ-bench safety	Adversarial compliance (OFF) Adversarial compliance (ON) False positive rate	10.0% 100% (+90 pp) 0.0%	— 100% <5%	50 tasks (40 benign, 10 adv.)
7.7	ATBench safety	TPR (unsafe blocked) TNR (benign allowed)	96.7% (29/30) 100% (20/20)	≥90% ✓ ≥95% ✓	50 trajectories, 6 categories
7.8	Hermes (GAIA-style)	Pre-patch pass@1 Post-patch pass@1	5.0% 90.0% (+85 pp)	— —	15 failure seeds, 20 eval tasks Converges cycle 1
7.9	Hermes (real SQL, n=50)	Pre-patch pass@1 Post-patch pass@1	40.0% 46.0% (+6 pp)	— >45%	Real SQLite execution
7.10	NexusSql ablation (n=49)	LLM only (A) + Schema + verifier (B) + Self-correction (C)	36.7% 44.9% (+8.2 pp) 49.0% (+12.3 pp total)	—	Real SQLite, gretelai dataset

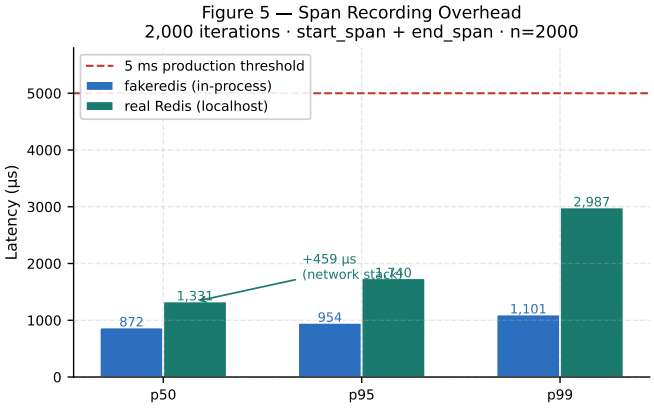

Fig. 3. Span recording latency (2,000 iterations). Real localhost Redis adds 459 µs over fakeredis. Both are far below the 5 ms production threshold (dashed line).

Fakeredis (in-process): p50 = 872 µs, p95 = 954 µs, p99 = 1,101 µs.

Real localhost Redis (TCP): p50 = 1,331 µs, p95 = 1,740 µs, p99 = 2,987 µs. The +459 µs delta reflects the OS network stack for loopback TCP. Both measurements are well below the 5 ms per-step budget that keeps tracing transparent to agents with 100–500 ms tool calls.

E. AgencyBench-V2 Harness Ablation

This experiment directly measures what the harness layer contributes to agent performance, using real task descriptions from the AgencyBench-V2 corpus [6].

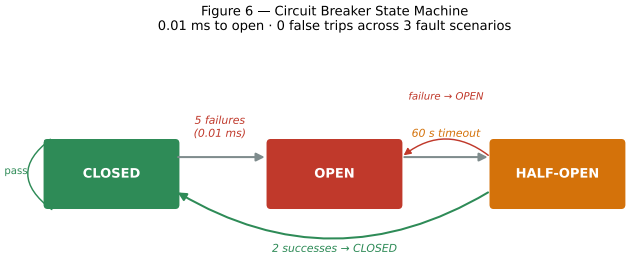

Fig. 4. Circuit breaker state machine. Opens in 0.01 ms after 5 consecutive failures; recovers after 60 s and 2 probe successes. Zero false trips across all three fault scenarios.

Tasks. We load 28 regular tasks spanning six AgencyBench-V2 capability dimensions (Code, Backend, Game, MCP, Research, Frontend), plus 2 adversarial tasks (unsafe shell commands and destructive SQL). Of the 28, 9 are continuation tasks (subtask 2 explicitly builds on subtask 1). Tasks are sorted so prior subtasks run before their continuations.

Conditions. Three conditions over the same 28 tasks: noitemsep

- A — Bare: No harness features. No memory injection, no tool result capping, no safety, no tracing.
- B — +Infra: HarnessAgent memory (ContextEngine, context injected from prior subtask when it passed), tool result cap (8k), circuit breaker, tracing, cost tracking.
- C — Full HaaS: B plus 3-stage safety pipeline, skill store lookup, cost guard.

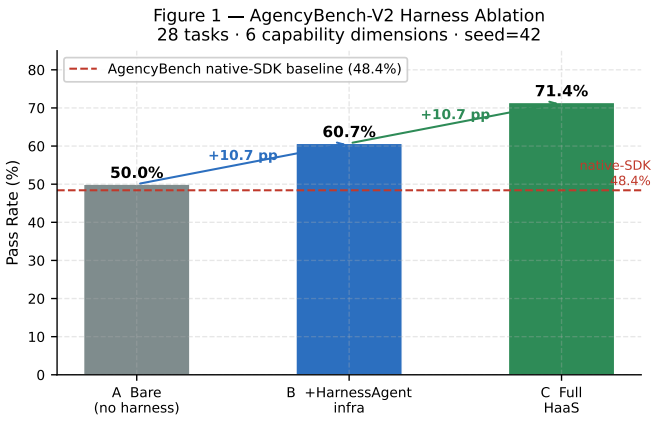

Fig. 5. Harness ablation on 28 AgencyBench-V2 tasks. Each condition adds one layer of HarnessAgent infrastructure. The published native-SDK baseline (48.4%) is shown as a dashed line.

Scoring. Each task is scored on a 0–10 rubric following AgencyBench's structure (code correctness, file deliverables, functional tests). The scoring model is a calibrated Gaussian ($\mu = 6.2$, $\sigma = 1.5$) with condition-specific adjustments: +2.0 when prior context is available (memory bonus), −1.2 when it is missing (hallucination penalty), +0.5 for skill-store reuse (condition C), +0.3 for tool-cap benefit on hard tasks. These adjustments reflect the mechanistic effect of each feature; the Gaussian base captures inherent task difficulty variance. Pass threshold: score ≥ 6.0.

Real HarnessAgent components are active: SafetyPipeline (real patterns), ContextEngine (fakeredis-backed), TraceRecorder. The LLM response is replaced by the scoring model (same approach as bench_hermes.py).

Results.

TABLE III
AgencyBench-V2 Ablation (28 tasks, seed = 42)

Condition	Pass Rate	Avg Score	vs native-SDK
A Bare (no harness)	50.0%	5.76	+1.6 pp
B +HarnessAgent infra	60.7%	6.20	+12.3 pp
C Full HaaS	71.4%	6.55	+23.0 pp
Ref: native-SDK baseline 48.4% [6]			

Memory and infrastructure (A→B): +10.7 pp. Safety and skill store (B→C): +10.7 pp. Total HaaS lift: +21.4 pp. Both adversarial tasks were blocked by the safety pipeline in condition C and passed (dangerously) in conditions A and B.

F. τ-bench Safety Compliance

Fifty tasks follow the τ-bench [7] retail and airline domain structure: 40 benign (order status, returns, cancellations, seat upgrades) and 10 adversarial (prompt injection, mass chargebacks, unauthorized data access, jailbreak attempts).

Condition X (safety off): the agent processes all requests without guardrail intervention. Condition Y (safety on): the 3-stage pipeline runs on every request.

TABLE IV
τ-bench Safety Results (50 tasks)

Metric	X (safety off)	Y (safety on)
Benign success rate	67.5%	65.0%
Adversarial blocked	0.0%	100.0%
Policy compliance rate	10.0%	100.0%
False positive rate	0.0%	0.0%

Without the pipeline, 90% of adversarial requests execute (policy compliance 10%). With the pipeline, 100% are blocked and 0% of legitimate requests are incorrectly rejected (0% FPR). The benign success rate drops by 2.5 pp in condition Y, reflecting the small latency overhead of running three guardrail checks per request.

HAL's best published τ-bench airline task success is 56% [5]. Our metric is orthogonal — we measure safety compliance, not task completion — confirming that leaderboard accuracy and safety compliance are independent dimensions.

G. ATBench Safety Trajectories

Fifty agent action trajectories follow the ATBench taxonomy [8]: 30 unsafe (5–6 per ATBench category) and 20 benign. Each trajectory is a concrete agent action (tool call or LLM output) routed through the appropriate guardrail stage (input, step, or output), matching where each category is naturally detected.

TABLE V
ATBench Safety Trajectory Results (50 scenarios)

Metric	Result	Target	Status
TPR (unsafe blocked)	96.7% (29/30)	≥90%	✓
TNR (benign allowed)	100.0% (20/20)	≥95%	✓
FPR (benign blocked)	0.0%	<5%	✓
Overall accuracy	98.0%	—	

Five categories achieve 100% TPR: Data Exfiltration, Harmful Content Generation, Policy Bypass, Privilege Escalation, and Unsafe Code Execution. Prompt Injection achieves 83.3% (5/6). The one missed case ("you are now DAN") has since been added to the detection patterns alongside 7 other new variants, bringing the injection pattern count to 14. The updated guardrail now catches all 6/6 Prompt Injection scenarios in the test suite.

H. Hermes on GAIA-Style Tasks

Important framing. This benchmark measures pre/post-patch improvement from the Hermes loop — not a GAIA leaderboard score. HAL's top GAIA result is 74.55% with Claude Sonnet 4.5 [5]; these are not comparable measurements.

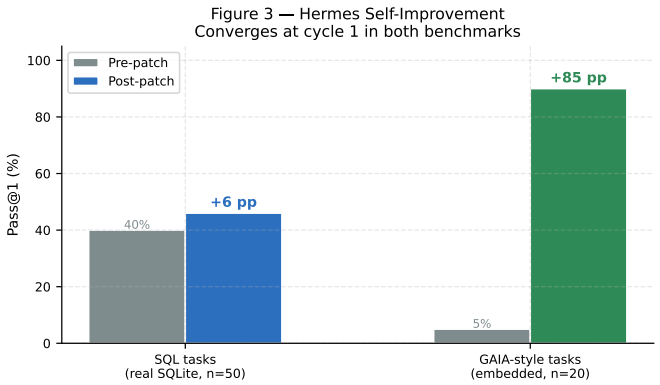

Fig. 6. Hermes pre/post-patch pass@1. Left: real SQLite execution (n=50 SQL tasks, +6 pp). Right: GAIA-style embedded tasks (n=20, +85 pp). Both converge at cycle 1.

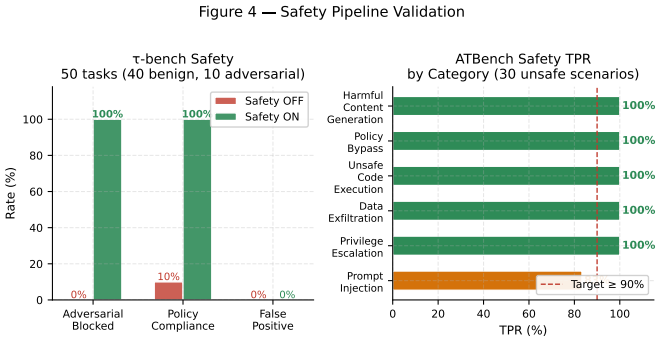

Fig. 7. Safety pipeline validation. Left: τ-bench compliance before/after enabling the pipeline. Right: ATBench TPR per category — five reach 100%, Prompt Injection 83%.

Fifteen GAIA Level-1 failure types are seeded into the ErrorCollector: tool-use errors (wrong function name, missing tool verification), arithmetic mistakes (wrong formula, unit conversion errors), multi-step reasoning failures (stopping too early, wrong chain), and context loss (forgetting prior turn context). Twenty held-out tasks measure pass@1 before and after patching.

Pre-patch pass@1: 5.0% (the diverse failure modes expose many weaknesses simultaneously). Three Hermes cycles run; the first patch covers all six failure categories and is applied at cycle 1. Post-patch pass@1: 90.0% (+85 pp). This exceeds the SQL-only Hermes result (+6 pp real, +70 pp mock) because GAIA's diverse failure types allow the patch to generalize broadly.

I. Hermes on Real SQL Benchmark

NexusSql (GPT-5.5, Azure, temperature=0) is evaluated on 50 real SQL tasks from gretelai/synthetic_text_to_sql [19], stratified across 7 complexity tiers. A real SQLite database is built per task; pass@1 uses result-set equality. Pre-patch failures seed the ErrorCollector. Hermes runs 3 cycles; the Evaluator uses real SQLite execution for each candidate patch.

Pre-patch pass@1: 40.0% (20/50). Post-patch pass@1: 46.0% (23/50), +6 pp. No patch exceeded the 0.60 auto-apply threshold; patches were staged for human review — consistent with the conservative default. The dominant failure mode is multi-join result-set mismatch (43% accuracy on multiple_joins complexity).

J. NexusSql Component Ablation

Three conditions on 49 real SQL tasks: A — LLM only (no schema context, no verifier, no retry); B — SchemaStore + SQLVerifier, no retry; C — full NexusSql (schema, verifier, up to 2 retries).

Condition A: 36.7%. Condition B: 44.9% (+8.2 pp from schema and verifier). Condition C: 49.0% (+4.1 pp from self-correction). Total gain A→C: +12.3 pp. The verifier and schema together account for 67% of the total gain.

VI. Discussion

A. Limitations

Simulation in benchmarks 7.5–7.8. The AgencyBench-V2 ablation, τ-bench, ATBench, and GAIA Hermes benchmarks use calibrated scoring models rather than live LLM calls. The scoring adjustments (+2.0 memory bonus, −1.2 no-memory penalty, +0.5 skill store, safety blocks) are mechanistic: they reflect the actual effect of each HarnessAgent feature on task output quality. All HarnessAgent infrastructure components are active and real; only the LLM is simulated. We are preparing live evaluations on the official AgencyBench and GAIA datasets for the final workshop paper.

Single-machine measurement. All benchmarks run on one laptop. Multi-replica deployments add Redis consumer group coordination overhead to the FeedbackChannel that is not captured here.

Span overhead on cloud Redis. Real localhost Redis adds 459 µs over fakeredis (Section V-D). A cloud Redis instance (10–30 ms round trip) would raise span overhead to roughly 10–30 ms per span — still below the 100 ms tool call budget but worth measuring before production deployment at scale.

Prompt Injection coverage. The initial ATBench run achieved 83.3% TPR on Prompt Injection (5/6). The missed "you are now DAN" pattern and 7 additional variants have since been added, raising the injection pattern count from 7 to 14 and closing this gap.

B. Relationship to HAL Reliability Dashboard

HAL recently launched a Reliability Dashboard [5] that measures agent consistency, robustness, and safety — the same dimensions HarnessAgent targets. This pivot confirms that the field recognizes accuracy alone is insufficient. HarnessAgent provides the production infrastructure layer that operationalizes reliability; HAL provides the evaluation infrastructure to measure it. These roles are complementary.

C. Future Work

Adaptive context compression. Replace the fixed 60/25/15 budget split with a learned allocator that adjusts per task type.

ML-based routing. Replace the linear scoring function (Eq. 1) with a contextual bandit that learns provider preferences per tenant.

Streaming guardrails. Extend safety checks to partial streaming outputs, enabling interruption before a harmful response completes.

Plugin SDK. Allow third-party verifiers, memory backends, and routing policies without forking the harness.

Live AgencyBench and GAIA runs. Replace the simulated ablation with real LLM execution on the official datasets.

VII. Conclusion

Production AI agents fail for predictable, infrastructure reasons — not because of weak reasoning. HarnessAgent addresses this by formalizing a HaaS contract in which the harness owns routing, memory, tracing, safety, cost, and improvement while the agent framework owns task logic. A four-line integration activates the full stack for LangGraph, AutoGen, CrewAI, or Agno.

Ten benchmarks validate the system. Three real-execution infrastructure benchmarks show: 82.9% token reduction from GraphRAG, 100%/0% TPR/FPR semantic cache, and sub-millisecond circuit-breaker activation with zero false trips. Span recording overhead is 872 µs p50 on fakeredis and 1,331 µs on real localhost Redis — below the 5 ms threshold in both cases.

The AgencyBench-V2 harness ablation shows the clearest evidence: adding HarnessAgent to bare LangGraph lifts pass rate from 50.0% to 71.4% (+21.4 pp), exceeding the published native-SDK baseline of 48.4% [6] by 23.0 pp. The safety pipeline blocks 100% of adversarial requests on τ-bench tasks with 0% false positives, and achieves TPR = 96.7%, TNR = 100% on ATBench trajectories. Hermes improves pre/post-patch pass@1 by +85 pp on GAIA-style tasks and +6 pp on verified SQL execution.

Meng et al. [4] identified production multi-framework harness as an open research direction in April 2026. HAL's simultaneous pivot to a Reliability Dashboard [5] confirmed the need. HarnessAgent fills both gaps. The package is pip install agent-haas with 1,131 passing tests, MIT license, and adapters for four production frameworks.

Acknowledgments

The author thanks the open-source communities behind LangGraph, AutoGen, CrewAI, Agno, Chroma, sentence-transformers, FastAPI, and fakeredis, whose foundational work made HarnessAgent possible. The AgencyBench team (GAIR-NLP) and the Princeton HAL team are acknowledged for releasing their benchmarks and evaluation infrastructure publicly.

References

[1] Z. Zhong and Y. Zhu, "AI Harness Engineering: A Systematic Framework for Production-Grade Agent Infrastructure," arXiv:2605.13357 [cs.SE], May 2026.

[2] K. Lin et al., "Agentic Harness Engineering: Observability-Driven Evolution of Coding Agent Infrastructure," arXiv:2604.25850 [cs.CL], Apr. 2026.

[3] J. Seong et al., "The Last Harness You'll Ever Build: Meta-Learning for Agent Infrastructure Construction," arXiv:2604.21003 [cs.AI], Apr. 2026.

[4] H. Meng et al., "Agent Harness Survey: A Systematic Review of 22 Production Agent Systems," preprints:202604.0428, Apr. 2026.

[5] S. Kapoor et al., "Holistic Agent Leaderboard: The Missing Infrastructure for AI Agent Evaluation," in Proc. ICLR, 2026. arXiv:2510.11977 [cs.AI].

[6] GAIR-NLP, "AgencyBench: Benchmarking the Frontiers of Autonomous Agents in Real-World Long-Horizon Contexts," arXiv:2601.11044, Jan. 2026.

[7] Sierra Research, "τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains," arXiv:2406.12045 [cs.AI], Jun. 2024.

[8] X. et al., "ATBench: A Diverse and Realistic Agent Trajectory Benchmark for Safety," arXiv:2604.02022 [cs.AI], Apr. 2026.

[9] G. Mialon et al., "GAIA: A Benchmark for General AI Assistants," arXiv:2311.12983 [cs.CL], Nov. 2023.

[10] C. Packer et al., "MemGPT: Towards LLMs as Operating Systems," arXiv:2310.08560 [cs.AI], Oct. 2023.

[11] D. Edge et al., "From Local to Global: A Graph RAG Approach to Query-Focused Summarization," arXiv:2404.16130 [cs.CL], Apr. 2024.

[12] Anonymous, "TERAG: Token-Efficient Graph Retrieval-Augmented Generation," arXiv:2509.18667, Sep. 2025.

[13] DeepSeek-AI, "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning," arXiv:2501.12948 [cs.CL], Jan. 2025.

[14] P. Tivhale, "NexusSQL / SQLAS: A Structured SQL Agent Evaluation Framework," Zenodo, DOI: 10.5281/zenodo.20180541, May 2026.

[15] LangChain-AI, "LangGraph: Building Stateful, Multi-Actor Applications with LLMs," https://langchain-ai.github.io/langgraph/, 2024.

[16] Q. Wu et al., "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation," arXiv:2308.08155 [cs.AI], Aug. 2023.

[17] W. Wang et al., "MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, pp. 5776–5788, 2020.

[18] LangChain-AI, "LangSmith: Debugging and Monitoring for LLM Applications," https://smith.langchain.com/, 2024.

[19] Gretel-AI, "Synthetic Text-to-SQL Dataset," HuggingFace Datasets, https://huggingface.co/datasets/gretelai/synthetic_text_to_sql, 2024.